Here is a **complete but brief guide to Strings in C++**, covering **all important topics** step-by-step. This is very useful for **C++ exams, interviews, and real coding**.

1️⃣ What is a String in C++?

A **string** is a sequence of characters used to store text.

In C++, strings are handled in **two ways**:

1. **C-style strings** (`char array`)
2. **C++ String class** (`std::string`)

2️⃣ C-Style String (Character Array)

🔹 Declaration

```
char name[20];
char city[] = "Dhaka";
```

🔹 Input & Output

```
cin >> name;           // stops at space
cin.getline(name,20); // reads full line
cout << name;
```

🔹 Important Functions (`<cstring>`)

```
strlen(str)    // length
strcpy(a,b)    // copy
strcat(a,b)    // concatenate
strcmp(a,b)    // compare
```

⚠️ Limitations

- Fixed size
- Error-prone
- No automatic memory management

3️⃣ C++ String Class (`std::string`) ✅ (Recommended)

🔹 Header

```
#include <string>
```

🔹 Declaration

```
string name;
string country = "Bangladesh";
```

4️⃣ Input & Output

```
cin >> name;            // single word
getline(cin, name);     // full line
cout << name;
```

5️⃣ String Length

```
name.length();
name.size();   // same as length
```

6️⃣ String Concatenation

```
string a = "Hello";
string b = "World";

string c = a + " " + b;
a += b;
```

7️⃣ String Comparison

```
if(a == b)  // equal
if(a != b)  // not equal
```

```
if(a > b)   // lexicographical
```

8️⃣ Access Characters

```
name[0];
name.at(1);
```

9️⃣ Modify String

```
name.push_back('A');
name.pop_back();
```

🔟 Substring

```
string s = "Programming";
cout << s.substr(0, 4);  // Prog
```

1️⃣1️⃣ Find Function

```
s.find("gram");  // returns index
```

1️⃣2️⃣ Insert & Erase

```
s.insert(3, "ABC");
s.erase(2, 4);
```

1️⃣3️⃣ Clear & Empty

```
s.clear();
s.empty();   // returns true/false
```

1️⃣4️⃣ Loop Through String

```
for(int i=0;i<s.length();i++)
    cout << s[i];

for(char c : s)
    cout << c;
```

1️⃣5️⃣ Convert String ↔ Number

```
int x = stoi("123");
string y = to_string(456);
```

1️⃣6️⃣ String with Functions

```
string getName(){
    return "AtiAr";
}
```

1️⃣7️⃣ String vs Char Array

Feature	char array	string
Size	Fixed	Dynamic
Safety	❌	✅
Functions	Limited	Rich
Easy	❌	✅

1️⃣8️⃣ Common Mistakes

❌ Forgetting `getline` after `cin`
❌ Accessing invalid index
❌ Mixing `char[]` and `string` incorrectly

1️⃣9️⃣ Best Practice ✅

✔ Always use `std::string`
✔ Use `getline` for sentences
✔ Use `.size()` instead of manual counting

🔚 Summary

- Use **`std::string`** in modern C++
- Avoid C-style strings unless necessary
- String class is **safe, flexible, and powerful**

---

summary write

1. Capacity
    1. s.size() => return the size of the string
    2. s.max_size() => returns the maximum size that string can hold.
    3. s.capacity() => returns the current capacity of the string
    4. s.clear() => clear the string
    5. s.empty() => return true/false if the string is empty
    6. s.resize() => change the size of the string
2. Element Access
    1. s[i] => access the `ith` index of the string
    2. s.at(i) => access the `ith` index of the string
    3. s.back() => access the last element of the string
    4. s.front() => access the first element of the string
3. Modifiers
    1. s+= => append the another string
    2. s.append() => append the another string
    3. s.push_back() => add character to the last string
    4. s.pop_back() => remove the last character of the string
    5. s= => string assign
    6. s.assign() => assign string
    7. s.erase() => erase characters from the string
    8. s.replace() => replace the portion of the string
    9. s.insert() => insert a portion to a specific position.
4. iterators

1. s.begin() => pointer to the first element
2. s.end() => pointer to the next element after the last element of the string.

Below is a **clean, structured explanation of `std::string` functions** you listed, **with simple examples** so it's easy to understand and remember for exams & coding.

1️⃣ Capacity (String Size & Memory)

1. `s.size()` / `s.length()`

➡️ Returns **number of characters** in the string.

```
string s = "Hello";
cout << s.size();   // 5
```

---

2. `s.max_size()`

➡️ Returns **maximum possible size** a string can have (depends on system).

```
cout << s.max_size();
```

📌 Mostly used for **theoretical / internal limits**, not daily coding.

---

3. `s.capacity()`

➡️ Returns **allocated memory capacity**, not actual size.

```
string s = "Hello";
cout << s.capacity();
```

📌 Capacity ≥ size
📌 Grows automatically when string grows

---

4. `s.clear()`

➡️ Removes all characters (size becomes 0).

```
s.clear();
```

---

5. `s.empty()`

➡️ Checks whether string is empty.

```
if(s.empty())
    cout << "Empty";
```

Returns `true` or `false`.

---

6. `s.resize(n)`

➡️ Changes string size.

```
string s = "Hello";
s.resize(3);
cout << s;   // Hel
```

If increased:

```
s.resize(8, 'x'); // Helxxxx
```

2️⃣ Element Access (Access Characters)

1. `s[i]`

➡️ Access **ith index**, no bounds checking.

```
cout << s[0];   // H
```

⚠️ Unsafe if index is invalid.

---

2. `s.at(i)`

➡️ Safe access, throws error if index invalid.

```
cout << s.at(1);  // e
```

---

3. `s.front()`

➡️ First character.

```
cout << s.front();  // H
```

---

4. `s.back()`

➡️ Last character.

```
cout << s.back();   // o
```

3️⃣ Modifiers (Modify String)

1. `s +=`

➡️ Append another string.

```
s += " World";
```

---

2. `s.append()`

➡️ Append string.

```
s.append(" World");
```

---

3. `s.push_back(ch)`

➡️ Add **single character** at end.

```
s.push_back('!');
```

4. `s.pop_back()`

➡️ Remove last character.

```
s.pop_back();
```

---

5. `s =`

➡️ Assign string directly.

```
s = "C++";
```

---

6. `s.assign()`

➡️ Assign new value.

```
s.assign("Programming");
```

---

7. `s.erase(pos, len)`

➡️ Remove characters.

```
string s = "Programming";
s.erase(3, 4);   // Proing
```

---

8. `s.replace(pos, len, newStr)`

➡️ Replace part of string.

```
s.replace(0, 3, "Pro");
```

---

9. `s.insert(pos, newStr)`

➡️ Insert string at position.

```
s.insert(3, "ABC");
```

4️⃣ Iterators (Very Important 🔥)

👉 Iterators work like **pointers** to characters.

---

1. `s.begin()`

➡️ Points to **first character**.

2. `s.end()`

➡️ Points to **position AFTER last character**.

🔹 Example: Print string using iterator

```
string s = "Hello";

for(auto it = s.begin(); it != s.end(); it++){
    cout << *it << " ";
}
```

Output:

```
H e l l o
```

📌 `*it` → value at iterator
📌 `it++` → move to next character

🔹 Reverse using iterator

```
for(auto it = s.end()-1; it >= s.begin(); it--){
    cout << *it;
}
```

✅ Quick Summary Table

Category	Purpose
Capacity	size, memory control
Element Access	read characters
Modifiers	change string
Iterators	traverse string

🔹 `stringstream` in C++ (Complete but Easy Explanation)

`stringstream` is used to **convert data between strings and other data types** and to **parse strings** easily.

📌 Header:

```
#include <sstream>
```

1️⃣ What is `stringstream`?

`stringstream` is a class that treats a **string like a stream (cin / cout)**.

You can:

- Convert **string → int/float**
- Convert **int/float → string**
- Split strings using **space or delimiter**

2️⃣ Types of String Streams

Class	Purpose
`stringstream`	Input + Output
`istringstream`	Input only
`ostringstream`	Output only

3️⃣ Basic Example

```
#include <iostream>
#include <sstream>
using namespace std;

int main(){
    string s = "123";
    stringstream ss(s);

    int x;
    ss >> x;

    cout << x + 5;   // 128
}
```

✔ Converts string `"123"` → integer `123`

4️⃣ Convert Number → String

```
int x = 456;
stringstream ss;
ss << x;

string s = ss.str();
cout << s;   // "456"
```

5️⃣ Split String by Space (Very Common)

```
string s = "I love C++";
stringstream ss(s);

string word;
while(ss >> word){
    cout << word << endl;
}
```

Output:

```
I
love
C++
```

6️⃣ Split String with Custom Delimiter

```
string s = "10,20,30";
stringstream ss(s);

string token;
while(getline(ss, token, ',')){
    cout << token << endl;
}
```

7️⃣ Parse Multiple Data Types

```
string s = "100 3.14 C++";
stringstream ss(s);

int a;
double b;
string c;

ss >> a >> b >> c;

cout << a << " " << b << " " << c;
```

8️⃣ Clear & Reuse `stringstream`

```
stringstream ss;
ss << "123";
int x;
ss >> x;

ss.clear();      // clear state
ss.str("");      // clear content
```

9️⃣ `stringstream` vs `stoi()`

Feature	stringstream	stoi
Multiple values	✅	❌
Parsing mixed data	✅	❌
Simpler	❌	✅
Safe	✅	❌

🔟 Common Use Cases

✔ Input parsing
✔ Competitive programming
✔ File & data processing
✔ Splitting user input
✔ Converting values safely

⚠️ Common Mistakes

❌ Forgetting `#include <sstream>`
❌ Not clearing stream before reuse
❌ Expecting delimiter handling without `getline`

🧠 Best Practice

✔ Use `stringstream` for **complex parsing**
✔ Use `stoi`, `stod` for **simple conversion**

🔚 Summary

- `stringstream` works like `cin/cout` on strings
- Excellent for parsing and conversion
- Very common in **interviews & contests**